

Step 1: Archive Solutions in GitLab and Create a Merge Request

1. **Sign in to GitLab** (<https://gitlab.com/>) or create an account if you don't have one.
2. **Create a new repository** for your project.
 - Click on "**New project**".
 - Choose "**Create blank project**".
 - Provide a **project name** and select **visibility (Private or Public as required)**.
 - Click "**Create project**".
3. **Push your source code** to GitLab:
 - If using Git Bash/Terminal, navigate to your project folder and run:

```
git init
```

```
git remote add origin https://gitlab.com/YourGitLabUsername/YourRepoName.git
```

```
git add .
```

```
git commit -m "Initial commit"
```

```
git branch -M main
```

```
git push -u origin main
```

Create a Merge Request:

- Go to "**Merge Requests**" in GitLab.
- Click "**New Merge Request**".
- Select the **source branch** (your working branch) and **target branch** (main).
- Click "**Create Merge Request**" and add a description.
- Assign reviewers if required.

Step 2: Create GitLab CI/CD Pipeline

1. Inside your project root directory, create a file named **.gitlab-ci.yml**.
2. Add the following configuration to run **build and test**:



.gitlab-ci.yml

Step 3: Integrate SonarCloud for Code Quality Scan

1. **Sign up for SonarCloud** (<https://sonarcloud.io/>) and log in.
2. Click **"Create New Project"**.
3. Select **"Analyze a new project"** → Choose **GitLab** as the provider.
4. Select your project repository and click **"Set Up"**.
5. Get your **Sonar Token**:
 - Go to **"My Account"** → **"Security"**.
 - Generate a new token and **copy it**.
6. **Add SonarCloud Token to GitLab CI/CD Variables**:
 - Go to **GitLab Project** → **Settings** → **CI/CD**.
 - Expand **"Variables"** and click **"Add Variable"**.
 - Name: SONAR_TOKEN, Value: **Paste your SonarCloud token**.
7. **Trigger the CI/CD Pipeline**:
 - Push changes to your repository to start the pipeline automatically.
 - Check **"CI/CD Pipelines"** in GitLab.

Step 4: Configure Package/Container Registry Deployment

1. **Go to GitLab Project** → **Packages & Registries** → **Container Registry**.
2. **Enable Container Registry** if not already enabled.
3. **Modify .gitlab-ci.yml to publish package**:

```
yaml                                                                    Copy Edit

deploy:
  stage: deploy
  script:
    - echo "Deploying package to GitLab registry..."
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker build -t $CI_REGISTRY_IMAGE:latest .
    - docker push $CI_REGISTRY_IMAGE:latest
```

? Add GitLab Container Registry Variables:

- Go to **Settings** → **CI/CD** → **Variables**.
- Add:
 - CI_REGISTRY_USER → Your GitLab username.
 - CI_REGISTRY_PASSWORD → Your GitLab Access Token.

? **Push your changes** to run the pipeline and upload your package

Step 5: Add jtomInterview@gmail.com as a Member

1. Go to **GitLab** → **Your Project** → **Settings** → **Members**.
2. Click **"Invite Members"**.
3. Enter jtomInterview@gmail.com, select **"Owner"** as the role, and click **"Add Member"**.
4. **Add SonarCloud:**
 - Go to **SonarCloud** → **Your Organization** → **Members**.
 - Click **"Add a Member"** and enter jtomInterview@gmail.com.

Step 6: Verify Everything

- Ensure the **GitLab pipeline runs successfully**.
 - Check **SonarCloud reports** for code coverage.
 - Confirm that the **package is uploaded** to GitLab registry.
 - Verify that **jtomInterview@gmail.com** has access.
-

Error

When run

git remote add origin <https://gitlab.com/interview3314711/hrms.git>

```
PS D:\Interview\HRMS> git remote add origin https://gitlab.com/interview3314711/hrms.git
fatal: detected dubious ownership in repository at 'D:/Interview/HRMS'
'D:/Interview/HRMS' is owned by:
'S-1-5-32-544'
but the current user is:
'S-1-5-21-2540086807-1823769574-3229602858-1001'
To add an exception for this directory, call:

    git config --global --add safe.directory D:/Interview/HRMS
PS D:\Interview\HRMS>
```

Solve

git config --global --add safe.directory D:/Interview/HRMS

git remote add origin <https://gitlab.com/interview3314711/hrms.git>

git remote -v

Correct output

```
PS D:\Interview\HRMS> git config --global --add safe.directory D:/Interview/HRMS
PS D:\Interview\HRMS> git remote add origin https://gitlab.com/interview3314711/hrms.git
PS D:\Interview\HRMS> git remote -v
origin https://gitlab.com/interview3314711/hrms.git (fetch)
origin https://gitlab.com/interview3314711/hrms.git (push)
PS D:\Interview\HRMS>
```

Task Run : git add .

Error

```
PS D:\Interview\HRMS> git add .
error: open(".vs/HRMS/FileContentIndex/05756119-1f92-4601-b8c1-166bbc724ceb.vsidx"): Permission denied
error: unable to index file '.vs/HRMS/FileContentIndex/05756119-1f92-4601-b8c1-166bbc724ceb.vsidx'
fatal: adding files failed
PS D:\Interview\HRMS>
```

Solution

✅ Step 1: Exclude Unnecessary Files Using .gitignore

1. **Create a .gitignore file in the HRMS project root directory:**
 - Open **Visual Studio**.
 - Right-click on the **HRMS Solution** → Click **"Add"** → **"New Item"**.
 - Select **"Text File"**, name it **.gitignore** (make sure it doesn't have .txt at the end).
 - Click **"Add"**.
2. **Add the following content to .gitignore:**



.gitignore

✅ Step 2: Clear Git Cache and Retry Adding Files

Now, clear the cache and try adding files again:

```
git rm -r --cached .
```

```
git add .
```

```
git commit -m "Added .gitignore and removed unneeded files"
```

✅ Step 3: Push to GitLab

Now, push your project to GitLab:

```
git push -u origin main
```

Expected Outcome

- Unwanted files (e.g., .vs, bin, obj) will be ignored.
- Only necessary files will be added to Git.
- Push will go through without permission errors.

Error

error: failed to push some refs to 'https://gitlab.com/interview3314711/hrms.git'

✅ Step 1: Check the Remote Status

Run the following command to check if there are new changes on the remote branch:

```
git status
```

✅ Step 2: Pull the Latest Changes from GitLab

Run this command to **fetch and merge the latest changes** from the GitLab repository:

```
git pull origin main --rebase
```

```
git push origin main
```

✅ Step 3: Force Push (ONLY if Necessary)

If git pull gives merge conflicts or fails, you can **force push** (⚠️ Be careful, as this overwrites remote changes):

```
git push --force origin main
```

✅ Step 4: Verify on GitLab

1. Open **GitLab** and go to your **HRMS repository**.

2. Check if the files are uploaded successfully.

```
PS D:\Interview\HRMS> git status
On branch master
nothing to commit, working tree clean
PS D:\Interview\HRMS> ]
```

✅ Step 1: Check Your Current Branch

Since your branch is named master, but you likely need main, **check the existing remote branches**:

```
git branch -a
```

✅ Step 2: Rename master to main and Push

To sync with the remote repository, rename your branch to main:

```
git branch -M main
```

```
git push -u origin main
```

```
PS D:\Interview\HRMS> git status
On branch master
nothing to commit, working tree clean
PS D:\Interview\HRMS> git branch -a
* master
PS D:\Interview\HRMS>
```

✅ Step 1: Check Remote Branches

Run the following command to list **all remote branches**:

```
git branch -r
```

If you see origin/main, that means the remote branch is named main, while your local branch is still master.

✅ **Step 2: Rename Local master to main (If Needed)**

If your remote repository uses main and your local branch is still master, rename it:

```
git branch -M main
```

```
git push -u origin main
```

✅ **Step 3: If Push Still Fails, Set Upstream Branch**

If pushing still fails, set the correct upstream branch:

```
git push --set-upstream origin main
```

```
git push origin main
```

✅ **Step 4: If the Remote Already Has Commits**

If your remote repository already has commits, **pull the latest changes first**

```
git pull origin main --rebase
```

```
git push origin main
```

✅ **Step 5: If main Branch Doesn't Exist on GitLab**

If there is **no main branch on GitLab**, create it and push:

```
git push -u origin master
```

Then, in **GitLab → Repository → Branches**, rename master to main (if required)

Error

```
To https://gitlab.com/interview3314711/hrms.git
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://gitlab.com/interview3314711/hrms.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
PS D:\Interview\HRMS>
```

✅ Step 1: Pull the Latest Changes from GitLab

Before pushing, you need to **fetch and merge** the latest changes from the remote repository.

Run:

```
git pull origin main --rebase
```

This will:

- Fetch the latest changes from GitLab.
- Reapply your local changes on top of them.

✅ Step 2: Resolve Any Merge Conflicts (If Needed)

If you get a merge conflict:

1. Open **Visual Studio** or check the conflicting files in Git.
2. Manually fix the conflicts.
3. Add the resolved files back:

```
git add .
```

```
git rebase --continue
```